

Harness Engineering

학습 체크리스트 (1/2)

☐ 에이전트의 제어 성능과 지시 이행력을 향상시키는 하네스 엔지니어링의 개념 이해

☐ AGENTS.md, SKILLS.md, DESIGN.md의 개별 역할 및 하네스 문서 정의 방법 숙지

☐ AI 에이전트에 최적화된 행동 규칙과 환경 정보를 전달하는 AGENTS.md 작성 및 최적화

☐ 외부 도구 및 스킬 연동 명세를 수립하고 활용하는 SKILLS.md 적용 흐름 파악

학습 체크리스트 (2/2)

- ☐ 일관된 프리미엄 UI/UX 구현을 위한 디자인 가이드라인인 DESIGN.md 활용
- ☐ 모델 콘텍스트 프로토콜(MCP)의 탄생 배경과 클라이언트-서버 구조의 핵심 아키텍처 이해
- ☐ Stitch 플랫폼의 MCP 설정을 복제하여 OpenCode 클라이언트에 연결하고 정상 구동 확인

하네스 엔지니어링 개요

하네스 엔지니어링 (Harness Engineering)

AI 코딩 에이전트가 작동하는 작업 공간(리포지토리)에 규격화된 지침, 도구(Skill), 디자인 사양을 주입하여 제어 성능과 지시 이행력을 극대화하는 엔지니어링 기법

- **제어력 향상:** 에이전트가 리포지토리의 맥락을 정확히 인지하고 목적에 맞게 활동하도록 통제
- **하네스 파일 구성:** AGENTS.md, SKILLS.md, DESIGN.md 등 세 축의 문서 체계를 활용
- **일관성 확보:** 인프라 환경 명시와 사전에 정의된 도구/디자인 사양으로 안정적 결과 도출

AGENTS.md 에이전트 명세서

에이전트 명세서 (AGENTS.md)

AI 에이전트가 리포지토리 내에서 반드시 준수해야 할 정체성, 미션, 터미널 지침, 코딩 컨벤션을 기술한 기본 규칙 문서

- **페르소나 지정:** 특정 프레임워크(React, Next.js 등)에 특화된 시니어 에이전트 페르소나 설계
- **개발 환경 주입:** OS, 패키지 매니저 버전 등 개발에 필요한 기본 인프라 정보 명시
- **에이전틱 지침 수립:** 주석 정책, 리팩토링 규칙, 커밋 메시지 컨벤션 등 강제 규칙 지정
- **공식 리소스:** AGENTS.md 공식 사양 참고

AGENTS.md 피드백 루프

- 설명: 개발 흐름과 프로젝트 규모가 진척됨에 따라 지침을 점진적으로 강화하는 환류(Feedback) 모델입니다.
- 주요 전략:
 - 동적 규칙 갱신: 개발 프로세스의 진행 상황에 맞추어 지침을 지속적으로 수정 및 보완
 - 지침 최적화 요구: 필요할 때 에이전트에게 관련 문서를 재작성 및 갱신하도록 명시적으로 지시
 - 지속적 학습 축적: 축적된 지침을 통해 에이전트가 실무 워크플로우에 최적화된 형태로 안착

SKILLS.md 스킬 명세서

스킬 명세서 (SKILLS.md)

에이전트가 샌드박스 내부 또는 외부 환경에서 직접 호출하고 실행할 수 있는 커스텀 도구들의 목록과 명세 문서

- **도구 목록 관리:** 에이전트가 기본 생성 능력을 넘어 파일 가공, API 통신, 보안 감사를 수행하도록 보조
- **스킬 활용 확장:** 필요한 커스텀 동작을 명세화하여 에이전트의 터미널 수행 능력의 한계 돌파
- **공식 리소스:** [Agent Skills 공식 소개](#) 및 [Skills Reference](#) 참고

스킬의 획득 및 연동 기법

- **설명:** 에이전트가 사용할 도구(Tool)를 설계하고 외부 환경과 안정적으로 바인딩하는 방법론입니다.
- **도구 획득 방식:**
 - **Skill Creator 기법:** GUI 웹 도구 및 스킬 메이커를 활용해 요구 사항에 딱 맞는 커스텀 함수 설계
 - **npx 패키지 연동:** 이미 검증된 공통 스킬셋 패키지를 `npx` 명령어로 다운로드하여 즉각 바인딩
 - **맥락적 도구 호출:** 에이전트가 `SKILLS.md` 를 스스로 참고하여 적절한 시점에 도구를 호출하도록 지원

DESIGN.md를 통한 UI/UX 통제

디자인 사양서 (DESIGN.md)

애플리케이션의 핵심 디자인 시스템, 테마 컬러 토큰, 컴포넌트 레이아웃 규칙을 정밀하게 기록한 스타일 가이드 문서

- 디자인 왜곡 방지: 확정된 기획 레이아웃 사양 및 컬러 토큰 정보를 템플릿 형태로 고정 보관
- Stitch 컴포넌트 설계 반영: 구글의 AI 디자인 설계 툴에서 도출한 컴포넌트 설계를 명세화하여 이식
- 프리미엄 UI/UX 지향: 에이전트가 그라데이션, 유리 효과(Glassmorphism), 애니메이션을 빌드하도록 유도
- 참조 리소스: [Stitch Design.md Overview](#) 및 [GetDesign.md \(Awesome Design MD 포함\)](#)

Model Context Protocol (MCP) 개요

모델 컨텍스트 프로토콜 (Model Context Protocol)

AI 모델이 다양한 데이터 소스, 외부 개발 도구, 실행 환경과 표준화되고 안전하게 통신할 수 있도록 설계된 오픈소스 규약

- **통합 표준 프로토콜:** 파편화된 외부 도구 연동 규격을 하나로 묶는 업계 표준 개방형 프로토콜
- **안전한 통신:** 샌드박스 및 표준 채널을 통해 로컬 개발망과 AI 모델 간의 통신 보안 제공
- **참조 문서:** [MCP 공식 문서](#) 및 [Google Discover MCP](#)

MCP 클라이언트-서버 구조

- 설명: AI 에이전트 플랫폼과 외부 리소스 간의 안전하고 유기적인 결합을 가능하게 하는 표준 아키텍처입니다.
- 주요 구성 요소:
 - 호스트 (클라이언트): OpenCode와 같은 IDE/에이전트가 실행 흐름을 제어하는 주체 역할 수행
 - MCP 서버: 파일 시스템 탐색기, DB 커넥터, API 서버 등 실제 도구와 리소스를 제공하는 서비스
 - 동적 리소스 쿼리: 호스트가 MCP 서버에 질의하여 사용 가능한 도구 목록을 수집한 뒤 유기적으로 가동

Stitch MCP 설정 및 OpenCode 연동 절차

- **설명:** 구글 Stitch 디자인 허브의 MCP 프로토콜 설정을 OpenCode 클라이언트에 연결하는 단계입니다.
- **연동 과정:**
 1. Stitch 플랫폼 로그인 후 원하는 프로젝트 내 **내보내기** > **MCP** > **MCP 설정** 이동
 2. 연동할 클라이언트 목록에서 **OpenCode** 를 지정한 뒤 화면에 나타난 MCP 설정 JSON을 복사
 3. 로컬 프로젝트의 루트 디렉토리에 **touch ./opencode.json** 으로 빈 파일을 생성
 4. 생성된 파일 내부에 복사한 설정 JSON 코드를 누락 없이 붙여넣고 저장
 5. 로컬 터미널에서 **opencode** 실행 후 **/mcps** 명령어를 입력하여 정상 연동 여부 검증

OpenCode MCP 연동 파일 생성 및 검증

```
1  # 1. 로컬 프로젝트 루트에 설정 파일 생성
2  touch ./opencode.json
3
4  # 2. JSON 코드 입력 및 저장 완료 후 OpenCode 실행
5  opencode
6
7  # 3. 에이전트 세션 내에서 MCP 연동 상태 체크
8  /mcps
```